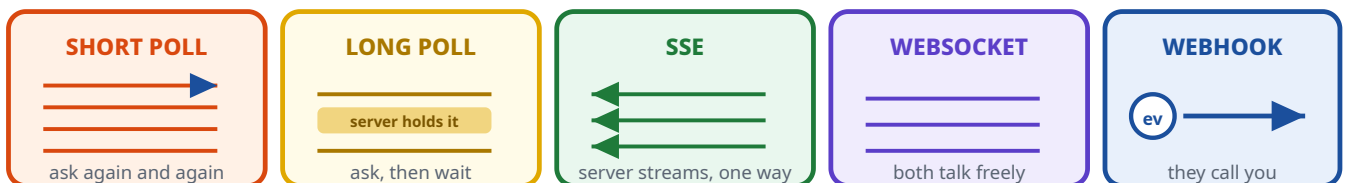


MODERN APPLICATION DEVELOPMENT II

Graded Assignment Complete Illustrated Solutions

Webhooks • Short polling • Long polling • Server-Sent Events • WebSockets • Pub/Sub •
Message brokers

Five ways to answer ONE question:
"how does new data reach me the moment it exists?"



All ten questions in this paper come from that one problem.

Plain HTTP has a rule: the client must speak first. The server is never allowed to start a conversation. These five techniques are five different ways around that rule.

Written for a complete beginner. Start at Part 1 and read straight through. Every idea gets a picture and an everyday story before any code appears.

READ THIS FIRST

How to use this book

This document does **two** jobs at once:

1. **It teaches the subject from zero.** Parts 1 to 8 build the knowledge brick by brick, with pictures and everyday stories.
2. **It solves all 10 questions**, showing exactly how a marker expects you to reason.

IMPORTANT NOTE ABOUT YOUR SCREENSHOTS

The screenshots you shared show **"Flag for review"** and **"Clear Selection"** buttons, and there are **no green "Answer is Correct" banners**. That means these are the options you had *selected* during the attempt, not confirmed results.

So every answer in this book was worked out independently from first principles and checked against the official documentation of each technology. The good news: **nine of your ten selections are correct**. The one to change is **Question 7, where option C should not be ticked**. Part 6 explains why in detail.

Map of the book

Part	What you learn	Questions solved
Part 1	The one problem behind all 10 questions: HTTP makes the client speak first	background
Part 2	Short polling: "are we there yet?" asked over and over	Q5
Part 3	Long polling: "hold the line, I will tell you when I know"	Q6, Q3
Part 4	Server-Sent Events: a one-way loudspeaker from server to browser	Q4
Part 5	WebSockets: a real phone call where both sides can talk	background
Part 6	Webhooks: "do not call us, we will call you"	Q1, Q2, Q7, Q10
Part 7	Pub/Sub and the message broker: the topic and the sorting office	Q8, Q9
Part 8	All five techniques in one comparison table	revision
Part 9	Answer key, cheat sheet, practice questions, glossary	revision

The boxes you will see

STORY BOX

A real-life picture of the idea. Read this one first, always.

DEFINITION BOX

The proper technical wording, the one to write in an exam.

TRAP BOX

The exact mistake that loses the mark. Learn these by heart.

ANSWER BOX

The final answer, with the option letters.

The one problem behind all 10 questions

Every question in this paper is a different answer to the same single problem. Understand the problem properly and the ten questions stop being ten separate facts to memorise.

1.1 The rule that causes all the trouble

STORY BOX • THE STRICT TEACHER

Imagine a classroom with a very strict rule: **the teacher is not allowed to speak unless a student asks a question first**. She can answer beautifully, in detail, as often as you like. But she can never call out "Ravi, your exam result just arrived!" She must wait to be asked.

That is exactly how plain **HTTP** works. The browser (client) must send a request first. Only then may the server reply. The server can **never** start the conversation.

The HTTP rule: the client must always speak first

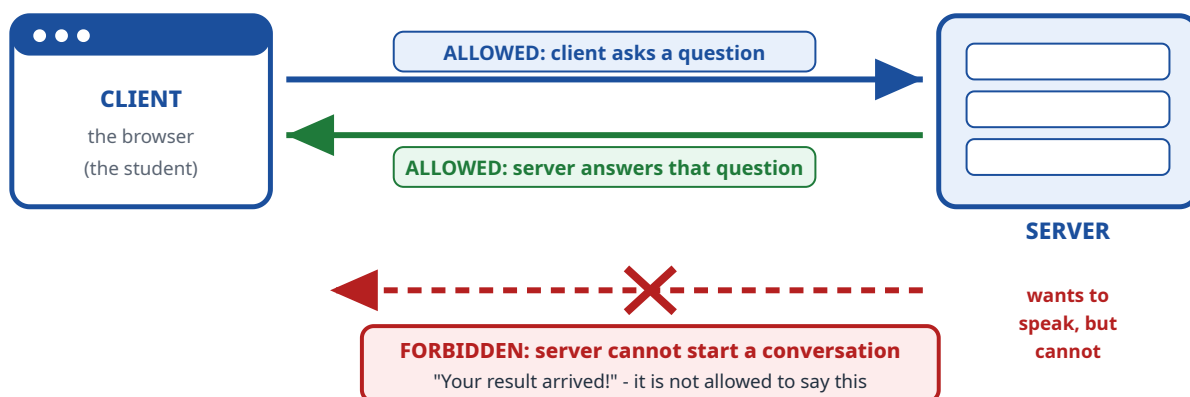


Figure 1.1 – HTTP is **client-initiated**. This single restriction is why all five techniques in this paper exist.

DEFINITION BOX

- **Pull** (also called client-initiated): the client asks, the server answers. A normal API call is a pull.
- **Push** (also called server-initiated): the data arrives without the receiver having just asked for it.

Plain HTTP only gives you pull. Every technique in this paper is a trick to get something that *feels* like push.

1.2 The five ways around the rule

Here is the whole paper on one page. Do not try to memorise it yet – just notice that the five techniques differ in only three things: **who starts**, **how long the connection stays open**, and **which direction data flows**.

The family of solutions, from clumsiest to cleverest

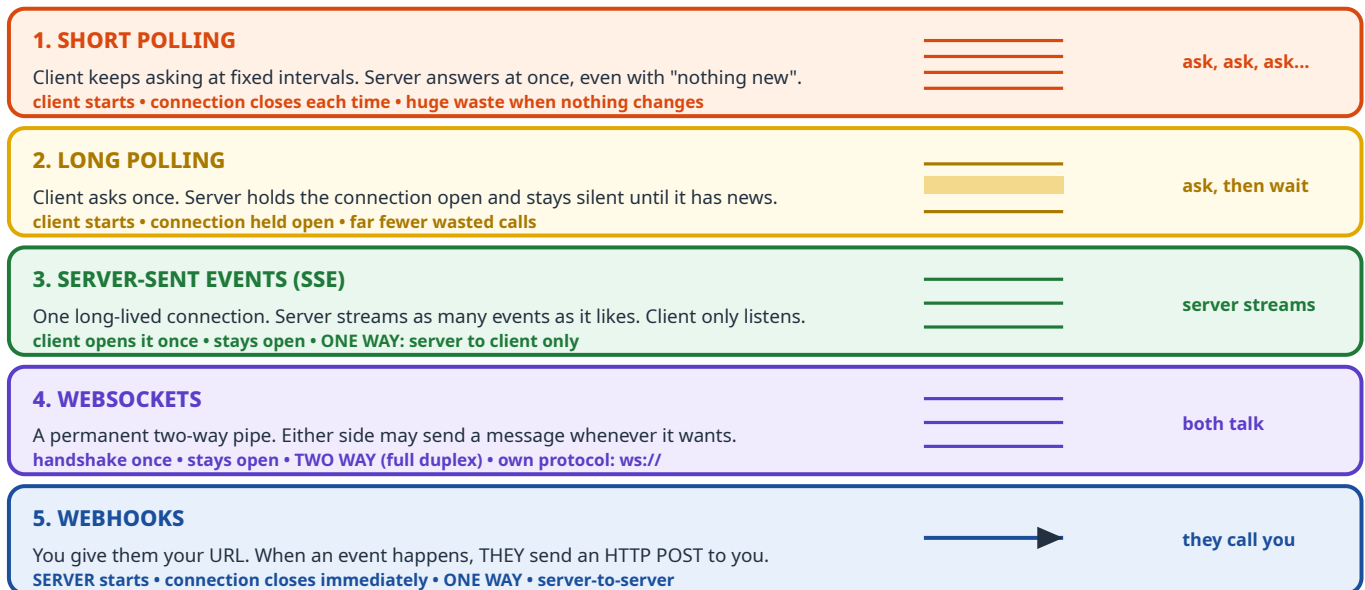


Figure 1.2 – The five techniques. Notice items 1 and 2 are client-started; items 3 and 4 are client-opened but server-driven; item 5 is fully server-started.

THE DISTINCTION THAT EARNS THE MOST MARKS IN THIS PAPER

Webhooks are for server-to-server. The other four are mainly for browser-to-server. A webhook needs the receiver to have a public URL that can accept an HTTP POST, which a browser tab does not have. That is why a webhook receiver is typically a small server application – for example a trivial Flask app.

Short polling

STORY BOX • "ARE WE THERE YET?"

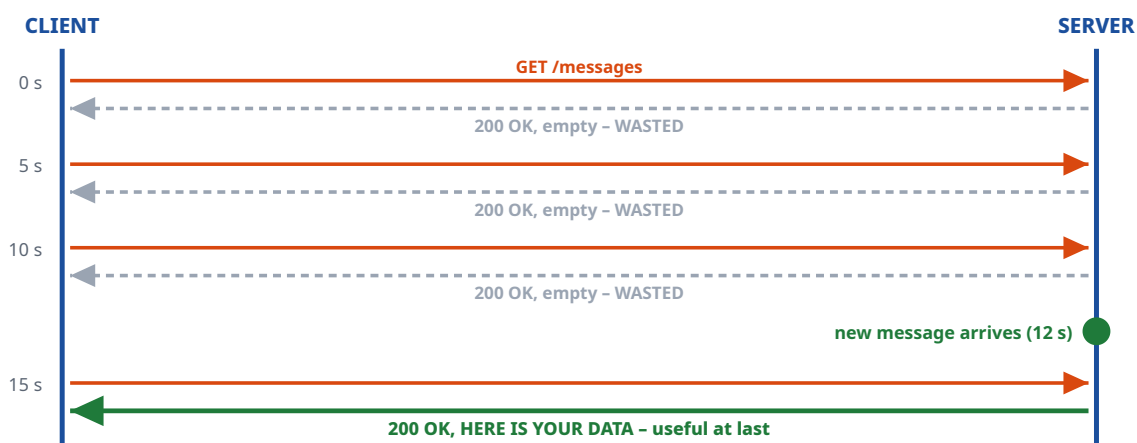
You are on a long car journey. Every two minutes you ask your father: "are we there yet?" and every time he answers immediately – usually "no, not yet". Occasionally, finally, "yes, we have arrived".

You got your answer eventually, but look at the cost: dozens of questions, and almost every single answer was useless. That is **short polling**, also just called **polling**.

DEFINITION BOX • SHORT POLLING

The client sends a request at **fixed, repeating intervals**. The server **replies immediately every time** – even when it has no new data, in which case it sends back an empty result. The connection is closed after each reply and opened fresh for the next one.

SHORT POLLING – ask every 5 seconds, get an instant answer every time



4 requests to get 1 piece of news, and the news was already 3 seconds old when it reached you.

Figure 2.1 – Short polling. Note the two costs: **wasted requests** and **stale data** (up to one whole interval late).

2.1 Why short polling is wasteful, in numbers

1000 users polling every 5 seconds, but news arrives only once an hour

requests sent per hour **720 000**

requests that carried news **1000**

99.86% of the traffic was for nothing.

Each empty reply still costs a TCP connection, HTTP headers, a database lookup and CPU on the server. This is why better methods were invented.

Figure 2.2 – The scaling problem. Short polling is simple to write but expensive to run.

Question 5

1 mark • select all that apply

Which of the following statement(s) is/are true regarding short polling?

- A.** The client sends a request to the server, and the server responds to the client immediately, even if it does not have data.
- B.** The client sends a request to the server, and the server waits to respond to the client if it does not have data and keep the connection open.
- C.** Request is sent after fixed delay periodically.
- D.** All of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	This is the definition of <i>short</i> . The server never keeps you waiting: if it has nothing, it says so straight away and closes the connection. Those are the grey dashed "empty" replies in Figure 2.1.
B	FALSE	Waiting with the connection held open is the definition of long polling, not short polling. This option belongs to Question 6.
C	TRUE	"Every 5 seconds" – a fixed, repeating interval. That is the "polling" part of the name.
D	FALSE	"All of the above" would need B to be true as well. Since B is false, D collapses. Also notice A and B are direct opposites, so they can never both be correct.

ANSWER TO QUESTION 5

A and C

Short polling = repeat at a fixed interval, and get an immediate reply each time, even an empty one. This matches the option you had selected.

TRAP BOX

Options A and B appear again, word for word, in Question 6 – but there the correct one flips. The examiner is checking one thing only: do you know which of the two waits? Fix it with this sentence:

"SHORT reply is short and instant. LONG polling makes you wait long."

Long polling

STORY BOX • "HOLD THE LINE"

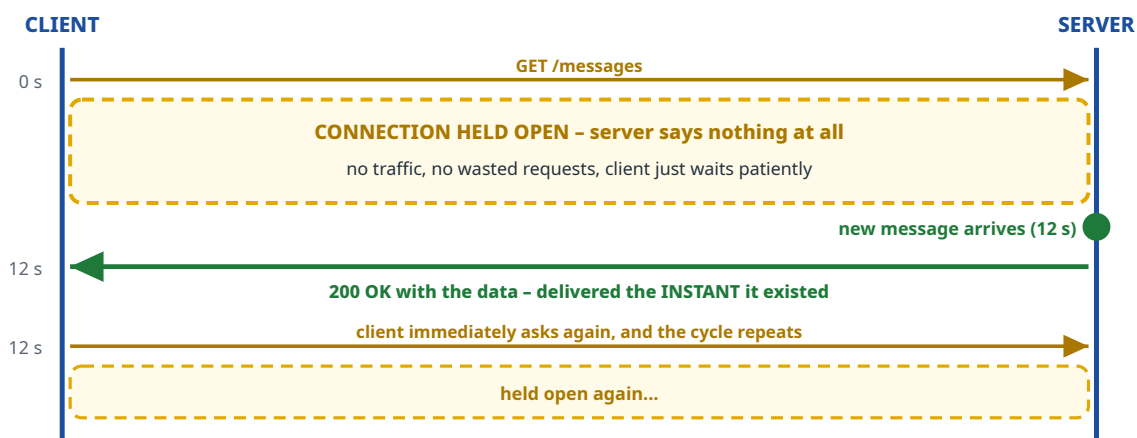
You ring the exam office and ask "are my results out?". Instead of saying "not yet, call back in five minutes", the clerk says: **"stay on the line, do not hang up. The moment they arrive I will tell you."**

You make *one* phone call instead of twenty. And the instant the news exists, you hear it – no waiting for your next scheduled call. That is **long polling**.

DEFINITION BOX • LONG POLLING

The client sends a request. If the server has no data, it **does not reply** – it **holds the connection open** and stays silent. As soon as data becomes available it sends the response and closes. The client then immediately opens a new request, and the cycle repeats.

LONG POLLING – ask once, and the server stays silent until it has news



1 request instead of 4, and zero delay: the news arrived at 12 s and reached you at 12 s.

Figure 3.1 – Long polling. Compare with Figure 2.1: the empty replies have vanished, and the data is fresh.

Question 6

1 mark • single correct

Which of the following is true regarding long polling?

- A. The client sends a request to the server, and the server responds to the client even if it does not have data.
- B. The client sends a request to the server, and the server waits to respond to the client if it does not have data and keep the connection open.
- C. Request is sent after fixed delay periodically.
- D. The client must send the next request immediately after it receives the response.

Option-by-option reasoning

Option	Verdict	Why
A	FALSE	Replying even with no data is short polling. This is the correct answer to Question 5, deliberately reused here as a distractor.
B	TRUE the answer	Exactly the definition: no data means no reply yet, and the connection is held open until there is something to send. The yellow dashed band in Figure 3.1.
C	FALSE	Fixed periodic delay is short polling again. In long polling the timing is decided by <i>when the event happens</i> , not by a clock.
D	FALSE	Careful: in practice the client usually <i>does</i> reopen straight away, but the word " must " makes this false. It is not a requirement of long polling – the client is free to wait, or to stop asking altogether. Compare with option A of Question 5, which is true because it describes what the mechanism <i>does</i> , not what it forces.

ANSWER TO QUESTION 6

Option B

Matches the option you had selected.

3.1 Short vs long polling, side by side

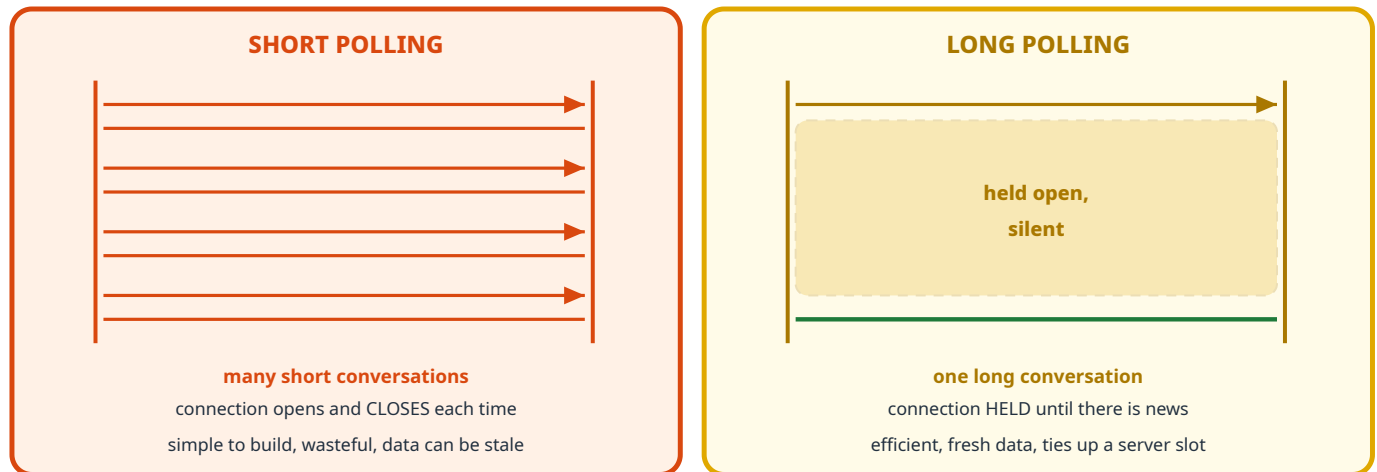


Figure 3.2 – The same 15 seconds, drawn both ways.

3.2 The "persistent connection" trap

Question 3 contains a subtle option about persistent connections. You must be precise here.

DEFINITION BOX

A **persistent connection** means one connection that is deliberately kept alive **across many messages**, as its normal way of working – like a WebSocket or an SSE stream.

Short polling has the **opposite** design: every poll is a brand-new, independent, short-lived request that is closed as soon as it is answered. So polling does **not require** a persistent connection. It works perfectly well without one – that is precisely why it is the crudest technique and also the easiest to deploy anywhere.

TRAP BOX • DO NOT CONFUSE THESE TWO

Long polling holds **one request** open while it waits for an answer. That is *not* the same as a persistent connection carrying *many* messages. And the statement in Q3 is about **polling** in general, which certainly does not require it.

Question 3

1 mark • select all that apply

Which of the following statement(s) is/are true regarding polling and long poll?

- ☒ A. In Polling, a client make repetitive calls to the server at fixed time intervals.
- ☐ B. Polling requires a persistent connection to the server.
- ☒ C. In long poll mechanism, the connection is kept open till the response is sent.
- ☐ D. All of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	"Repetitive calls at fixed time intervals" is the literal definition of polling. Figure 2.1.
B	FALSE	Polling closes the connection after every reply and opens a fresh one next time. A persistent connection is what SSE and WebSockets use, and it is the thing polling exists to avoid needing.
C	TRUE	"Kept open till the response is sent" is exactly the yellow band in Figure 3.1 – the defining feature of long polling.
D	FALSE	Needs B to be true. It is not, so D falls with it.

ANSWER TO QUESTION 3

A and C

Matches the options you had selected.

Server-Sent Events (SSE)

STORY BOX • THE RAILWAY STATION LOUDSPEAKER

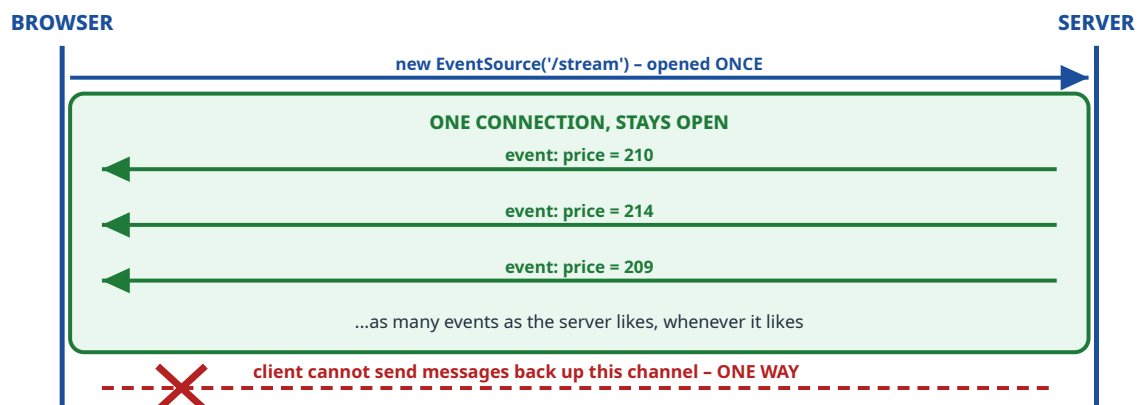
You reach the railway platform and switch on your ears. You do not ask the announcer anything. He simply keeps announcing, whenever there is something to announce: "train 5 minutes late" ... "platform changed to 3" ... "train arriving now".

You **listen**. You cannot talk back through the loudspeaker – it only works in one direction. That is **Server-Sent Events**: you open the channel once, then the server keeps pushing announcements down it for as long as you stay connected.

DEFINITION BOX • SSE

The client opens **one long-lived HTTP connection**. The server keeps it open and streams a sequence of events down it, in a special text format (content type `text/event-stream`). Communication is **one way only**: server to client. In the browser you consume it with the built-in `EventSource` object.

SERVER-SENT EVENTS – open once, then listen



Still ordinary HTTP underneath. No new protocol, no new port. That is SSE's big advantage over WebSockets.

Figure 4.1 – SSE. One connection, many events, one direction.

4.1 What SSE actually looks like in code

```
# SERVER: an ordinary Flask route that never stops yielding
from flask import Flask, Response
import time

app = Flask(__name__)

def event_stream():
    while True:
        price = get_latest_price()
        yield f"data: {price}\n\n"    # the SSE text format
        time.sleep(1)

@app.route('/stream')
def stream():
    return Response(event_stream(), mimetype='text/event-stream')
```

```
// CLIENT: three lines, using the browser's built-in EventSource
const source = new EventSource('/stream')

source.onmessage = (event) => {
    console.log('server says:', event.data) // fires again and again
}
```

NOTICE WHAT IS NOT IN THAT CODE

There is no message broker. No Celery. No Redis. No task queue. No `navigator.serviceWorker.register(...)`. SSE is just a Flask route that keeps yielding, and a browser object that keeps listening. Keep that in mind for the question below.

4.2 The three things people wrongly think SSE needs

Myth-buster: the ingredient list for SSE

SSE DOES need

- a long-lived HTTP connection kept open
- the mime type text/event-stream
- a client-side listener, i.e. EventSource
- a server that can hold many open connections

SSE does NOT need

- a task queue / message broker**
that is Celery + Redis territory, nothing to do with SSE
- a second protocol like ws://**
plain HTTP is enough
- the client to keep re-requesting**
unlike polling, it asks exactly once
- any request from client to server**
the channel is strictly one way

Figure 4.2 – A task queue is the classic wrong answer, because students mix SSE up with background jobs.

Question 4

1 mark • pick the FALSE one

Which of the following statements is **false** regarding server sent events?

- A. It is a mechanism at the server to push events to the client.
- B. It requires a task queue to be maintained.
- C. It requires the service workers on the client.
- D. The service workers should run in the background.

READ THE INSTRUCTION TWICE

This question wants the **false** statement. The option you tick is the *wrong* one. Underline the word "false" before you start.

Option-by-option reasoning

Option	True or false	Why
A	TRUE	That is the definition of SSE, word for word: a server-side mechanism for pushing events to the client. Figure 4.1.
B	FALSE the answer	A task queue plays no part in SSE. Look again at the code in section 4.1: a Flask route that yields, and a browser object that listens. Task queues (Celery, RabbitMQ, Redis as a broker) belong to <i>background job processing</i> , a completely separate topic. You can run SSE with no queue anywhere in your system.
C	TRUE as intended	The intended reading is "SSE needs something running on the client side to receive the events", which is correct – you must attach a listener, otherwise the stream arrives and nobody handles it. See the honesty note below about the exact wording.
D	TRUE	A client-side listener does have to keep running in the background while the page is alive, waiting for the next event. That is the whole point of a listener.

ANSWER TO QUESTION 4

Option B is the false statement.

SSE needs an open HTTP connection and a client-side listener. It does not need a task queue. This matches the option you had selected.

AN HONESTY NOTE ON OPTION C, SO YOU ARE NOT CAUGHT OUT LATER

Strictly speaking, a **Service Worker** is a specific browser technology used for offline caching and push notifications, and SSE does **not** use it – SSE uses the `EventSource` API running in the normal page. So the wording of option C is loose.

It does not change your answer, and here is the reasoning to use in an exam: the question asks for **one** false statement, and **B is unarguably false** – a task queue has nothing whatsoever to do with SSE. Options C and D are both about "something listening on the client", which SSE genuinely does require. So B remains the single best choice. When two options look shaky, always pick the one that is false *beyond any doubt*.

WebSockets

STORY BOX • THE WALKIE-TALKIE THAT NEVER SWITCHES OFF

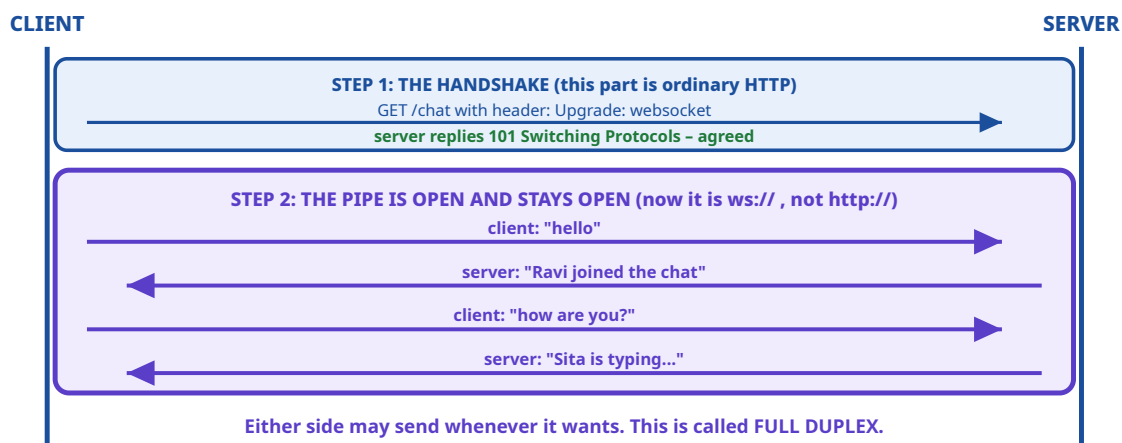
SSE was a loudspeaker: the announcer talks, you only listen. A **WebSocket** is a proper **phone call**. Once the call connects, either person can speak at any moment, as often as they like, without hanging up and dialling again.

This is why chat apps, multiplayer games and live collaborative editors use WebSockets: both sides genuinely need to send.

DEFINITION BOX • WEBSOCKET

A WebSocket starts as an ordinary HTTP request that asks to **"upgrade"** the connection. Once the server agrees, the connection switches to the **WebSocket protocol** (`ws://` or the secure `wss://`) and stays open. From then on it is **full duplex**: both client and server can send messages independently, at any time.

WEBSOCKET – one handshake, then both sides talk freely



Cost: a WebSocket needs server support for the protocol, and holds a connection per user.

Figure 5.1 – A WebSocket: HTTP only to shake hands, then a permanent two-way channel.

5.1 SSE vs WebSocket – the one difference that matters

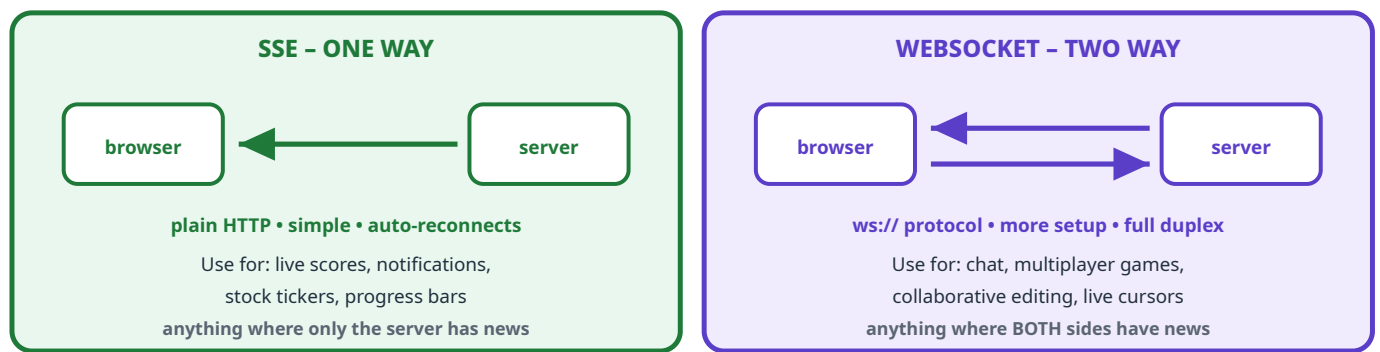


Figure 5.2 – Choose SSE unless the client genuinely needs to send messages too.

SAY IT OUT LOUD ONCE

"SSE is a loudspeaker: one way. A WebSocket is a phone call: two ways."

Webhooks

Four of the ten questions are about webhooks, so this is the part worth reading twice.

STORY BOX • "DO NOT CALL US, WE WILL CALL YOU"

You order a cake from a bakery. Two ways to find out when it is ready:

The polling way: you phone the bakery every ten minutes. "Ready?" "No." "Ready?" "No." Annoying for both of you.

The webhook way: you leave your **phone number** with them and go home. When the cake is ready, **they ring you**. One call, perfectly timed, zero waste.

A **webhook** is that phone number. You register a URL with someone else's system, and when an event happens, their server sends an HTTP POST to your URL. It is often described as a **reverse API**: instead of you calling them, they call you.

DEFINITION BOX • WEBHOOK

A webhook is a **user-defined HTTP callback**. You give a provider a public URL. When a chosen event occurs, the provider sends an **HTTP POST request** to that URL, carrying the event details in the **request body**. It is **event-driven**, **server-initiated**, and **one way**: they send, you acknowledge, the connection closes.

A WEBHOOK, END TO END

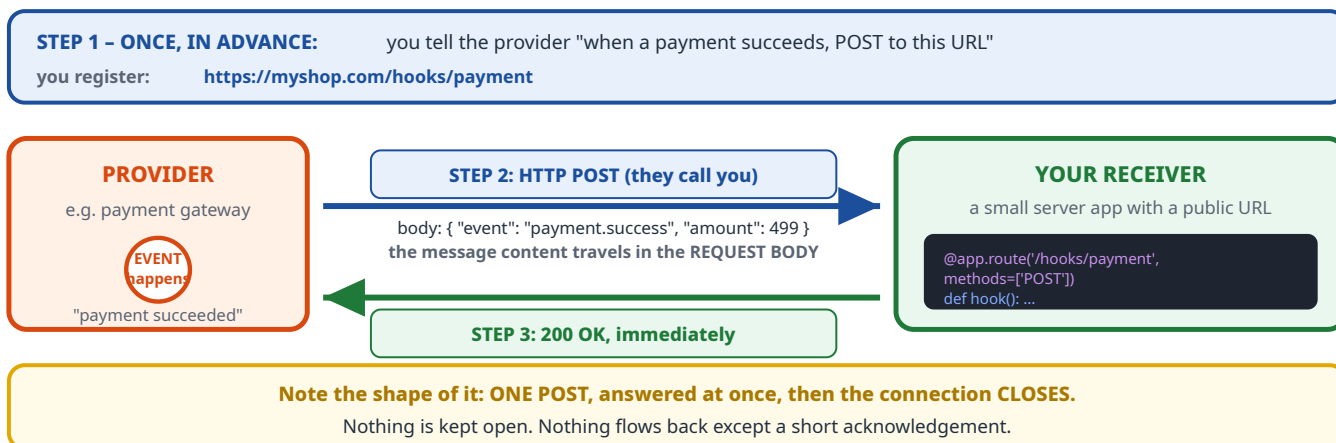


Figure 6.1 – A webhook is one HTTP POST triggered by an event, plus an immediate acknowledgement.

6.1 Why the receiver must answer immediately

STORY BOX • THE COURIER AT YOUR DOOR

A courier rings your bell. All he needs is your **signature** – five seconds. He does not want to stand there while you unpack the parcel, read the manual and assemble the furniture. If you make him wait, he marks the delivery as failed and comes back tomorrow.

A webhook sender is that courier. It wants a quick **200 OK** signature. If your receiver is slow, the sender **times out** and assumes delivery failed – and most providers then **retry**, so you can end up processing the same event several times.

Acknowledge first, work afterwards

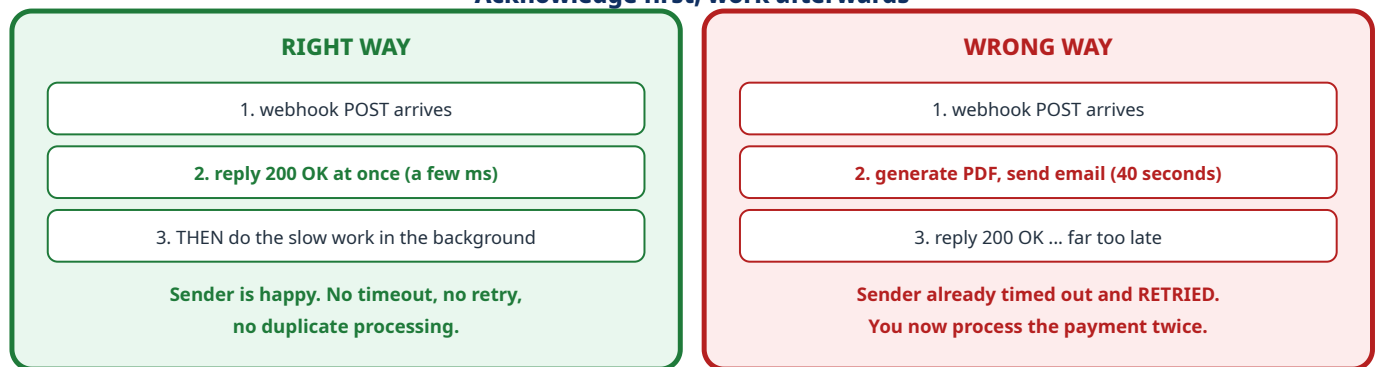


Figure 6.2 – This is why "a webhook should give an immediate response" is a true statement, and it is standard advice from every webhook provider.

6.2 Can a webhook be secured? Yes.

One option in Question 1 claims a webhook cannot be secured if its URL is public. That is wrong, and here is the standard toolkit that proves it.

Four normal ways to secure a publicly reachable webhook URL

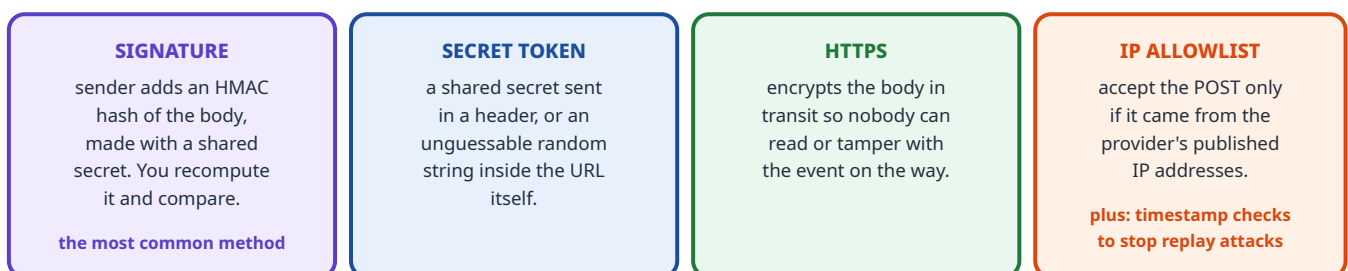


Figure 6.3 – "Publicly reachable" and "unsecured" are two completely different things.

THE IDEA TO CARRY AWAY

A URL being **reachable** by anyone does not mean it is **trusted** by anyone. Your front door is publicly reachable too – that is why it has a lock. The webhook's lock is the signature.

Question 1

1 mark • select all that apply

Which of the following statement(s) is/are true regarding Webhooks?

- ☒ A. A Webhook should give an immediate response.
- ☐ B. A Webhook cannot post form encoded data.
- ☐ C. A Webhook cannot be secured, if the receiver URL to be accessed is publicly available.
- ☒ D. A Webhook receiver can be a trivial flask application.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	The courier at your door (Figure 6.2). The receiver should acknowledge quickly with a 200 and do slow work afterwards, otherwise the sender times out and retries. This is the standard rule published by every webhook provider.
B	FALSE	The word cannot gives it away. A webhook is an ordinary HTTP POST, so its body can be any content type the two sides agree on. JSON (<code>application/json</code>) is the most common, but form-encoded (<code>application/x-www-form-urlencoded</code>) is perfectly normal and is what several well-known providers use. XML is also used.
C	FALSE	Publicly reachable is not the same as insecure. Signatures (HMAC), secret tokens, HTTPS and IP allowlists all secure a public webhook URL – see Figure 6.3. Every real payment gateway does exactly this.
D	TRUE	A receiver only has to accept a POST and return 200. That really is a handful of lines of Flask: <code>@app.route('/hook', methods=['POST'])</code> . Nothing more is required.

ANSWER TO QUESTION 1

A and D

Matches the options you had selected.

TRAP BOX • THE KEYWORD RADAR

Both wrong options contain the word **cannot**. Absolute words – *cannot*, *only*, *always*, *never* – are nearly always the sign of a false statement, because real engineering tools are built to be flexible. Circle every absolute word before you choose.

Question 2

1 mark • select all that apply

Which of the following statement(s) is/are correct, regarding Webhooks and Web sockets?

- A. A Webhook uses HTTP protocol in general.
- B. A web socket is used to achieve 2-way communication.
- C. A Webhook is primarily used for server-to-server communication.
- D. A Webhook generally keeps the connection open.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	A webhook is literally an HTTP POST to a URL. No special protocol is involved – that is what makes webhooks so easy to adopt.
B	TRUE	Two-way (full duplex) communication is the defining purpose of a WebSocket. Figure 5.1.
C	TRUE	To receive a webhook you need a public URL that can accept an incoming POST . A browser tab has no such thing. So webhooks run between servers: the provider's server calls your server.
D	FALSE	This is the option that separates webhooks from WebSockets. A webhook is a single POST followed by an immediate response, and then the connection closes . Keeping the connection open is what WebSockets and SSE do, not webhooks.

ANSWER TO QUESTION 2

A, B and C

Matches the options you had selected. Only D is false, because a webhook closes its connection straight away.

THREE WORDS THAT SEPARATE THE TWO

Webhook: one-shot, closes, server-to-server.

WebSocket: long-lived, stays open, browser-to-server.

6.3 API vs webhook: pull vs push

Questions 7 and 10 both test this direction. Get the arrows right and both are free marks.

Who starts the conversation?

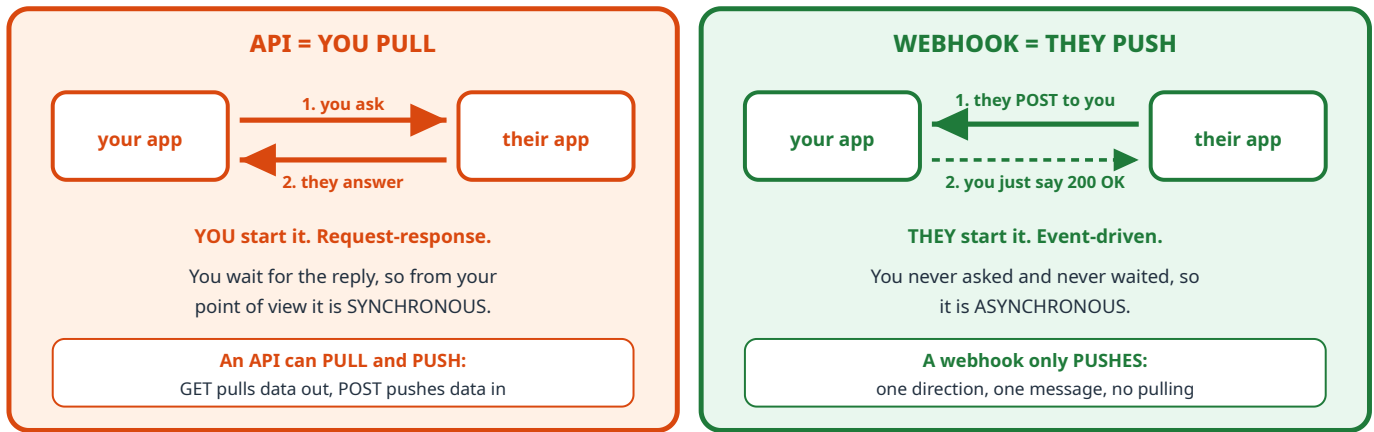


Figure 6.4 – The complete push/pull picture. An API is a shop you visit; a webhook is a home delivery.

Question 7

1 mark • select all that apply

Which of the following statement(s) is/are correct?

- ☒ A. A Webhook pushes the messages to an application.
- ☐ B. In general, webhook pulls the messages from an application, but an API pushes the messages to an application.
- ☐ C. Webhooks are generally synchronous in nature.
- ☒ D. An API can pull and push the messages from/to an application.

THIS IS THE ONE ANSWER TO CHANGE

Your screenshot has **A, C and D** ticked. Options A and D are right, but **option C should not be ticked**. The correct answer is **A and D**. The reasoning is below, and it is worth reading carefully because this exact distinction comes up again and again.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	A webhook pushes: the provider sends an HTTP POST into your application without being asked. Right-hand side of Figure 6.4.
B	FALSE	The two words are swapped round . Webhooks push, APIs pull. This option says the exact opposite of the truth, which is the most common trick in the whole topic.
C	FALSE change this	Webhooks are asynchronous in nature, not synchronous. They are the standard example of event-driven communication: the receiver never asks and never waits, it simply reacts whenever an event happens to arrive. Synchronous is the word for the <i>API</i> model, where the caller sends a request and blocks until the answer comes back. See the detailed explanation below.
D	TRUE	An API is two-directional in this sense: <code>GET</code> pulls data out of an application and <code>POST</code> or <code>PUT</code> pushes data into it. A webhook, by contrast, only ever pushes.

Why "webhooks are synchronous" is the wrong description

It is worth being precise, because there is a grain of truth that traps people.

Synchronous or asynchronous? Ask the right question

The grain of truth

One single webhook delivery is an HTTP request that waits for a 200 response. At that tiny transport level, yes, there is a request and a matching reply.

But that is true of literally every HTTP call ever made. It does not describe the NATURE of the webhook pattern.

What the exam means by "nature"

Does the receiver have to be waiting for this data at the moment it arrives?

No. It arrives whenever the event happens. That is ASYNCHRONOUS.

Same logic as Question 8: pub/sub is called asynchronous for exactly this reason, even though each delivery is a normal call.

Figure 6.5 – Judge the pattern, not the single HTTP call. Every reference describes webhooks as event-driven and asynchronous, in contrast to the request-response API model.

A CONSISTENCY CHECK YOU CAN DO INSIDE THE EXAM HALL

Question 8 of this very paper asks about pub/sub and marks "the communication is asynchronous in nature" as **correct** while "synchronous" is **wrong**. A webhook is the same shape of thing: an event happens, a message is pushed, the receiver reacts later. It would be inconsistent for the paper to call one asynchronous and the other synchronous. Use the paper against itself.

ANSWER TO QUESTION 7

A and D

Webhooks push and are asynchronous; APIs can both pull and push. **Do not tick C.**

Question 10

1 mark • select all that apply

Which of the statement(s) is/are true for webhook?

- ☒ A. The message content is sent as part of the request body.
- ☐ B. The message content in the response body should be detailed.
- ☐ C. Webhooks are usually invoked by the human clients.
- ☒ D. Webhooks are one way messages.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	The event details travel in the request body of the POST, typically as JSON. Figure 6.1, step 2.
B	FALSE	Backwards. The response should be as small and fast as possible – ideally just a bare 200 OK. The sender is not waiting to read a detailed report; it only wants the signature. A big, slow response risks the timeout in Figure 6.2.
C	FALSE	No human is involved. A webhook is triggered by an event inside a system – a payment settling, code being pushed, a file finishing upload. That is the meaning of "event-driven", and it also connects back to Q2: webhooks are server-to-server.
D	TRUE	One direction only: provider to receiver. There is no channel for the receiver to send messages back – the 200 OK is an acknowledgement, not a reply with content. Contrast with a WebSocket, which is two way.

ANSWER TO QUESTION 10

A and D

Matches the options you had selected.

THE WEBHOOK SENTENCE TO MEMORISE

"An event happens on their server, so their server POSTs the details in the **request body** to my public URL, and my app replies **200 immediately** and does the real work afterwards. One direction, then the connection closes."

Pub/Sub and the message broker

7.1 What is publish-subscribe?

STORY BOX • THE SCHOOL NOTICE BOARD

The head teacher has news: "sports day is on Friday". She does not walk to each of 600 students and tell them one by one. She does not even know who is interested. She pins **one notice** on the board under the heading "**SPORTS**".

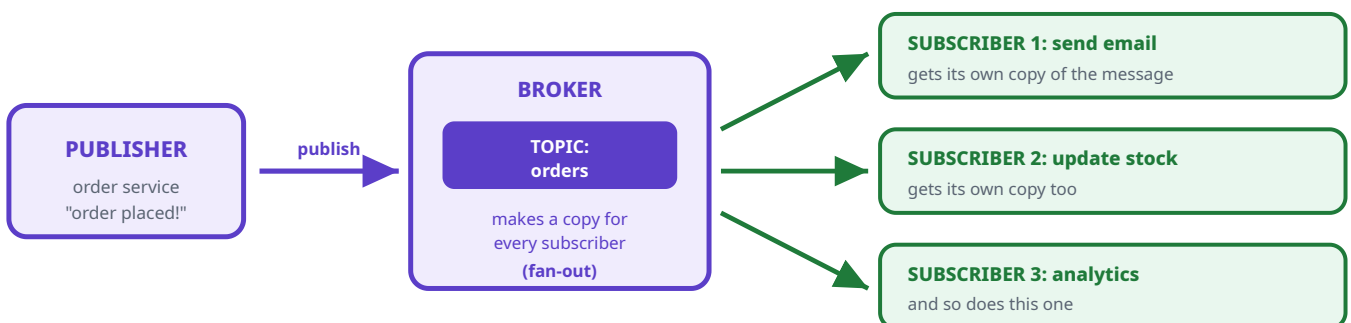
Everyone who cares about sports has already decided to watch that heading. All of them read the same notice. The head teacher never learns their names.

- The head teacher = the **publisher**.
- The heading "SPORTS" = the **topic**.
- The students watching that heading = the **subscribers**.
- The notice board itself = the **message broker**.

DEFINITION BOX • PUB/SUB

A **publisher** sends a message to a named **topic**, not to a particular receiver. The broker then delivers a **copy to every subscriber** of that topic. The publisher does not know who the subscribers are, how many there are, or whether there are any at all. This is called **fan-out**, and the not-knowing is called **decoupling**.

PUB/SUB – one message published, every subscriber gets a copy



Why this is the foundation of EVENT DRIVEN ARCHITECTURE

To add a "send SMS" feature tomorrow, you write a new subscriber and change NOTHING else. The publisher is never edited, never redeployed, and never even told. Services stay independent.

Figure 7.1 – Pub/Sub. One publish, three independent reactions.

7.2 Pub/Sub vs a plain message queue

	Message queue (point-to-point)	Pub/Sub (topic)
Who receives a message	exactly one consumer	every subscriber gets a copy
The question it answers	"who should <i>do</i> this job?"	"who should <i>know</i> this happened?"
Typical use	background work: resize this image	notifying many services of one event
Delivery style	consumer pulls when ready; message waits in the queue	broker pushes to subscribers as soon as it is published
Timing	asynchronous	asynchronous

THE POINT THAT QUESTION 8 TESTS

In pub/sub the broker's job is to hand the message to the topic's subscribers **straight away** – that is the whole idea of a topic. This "deliver it now to everyone listening" behaviour is what makes pub/sub suitable for live event notification, and it is entirely compatible with the communication being **asynchronous**: *asynchronous* means the **publisher does not stop and wait**, not that delivery is slow.

Question 8

1 mark • select all that apply

Which of the following is correct regarding pub/sub messaging?

- ☒ A. It can be used to enable event driven architecture.
- ☒ B. Any message published to a topic should be immediately received by the subscriber(s) of the topic.
- ☐ C. The communication is synchronous in nature.
- ☒ D. The communication is asynchronous in nature.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	Pub/Sub is the standard building block of event-driven architecture: publish an event, and any number of services react independently. Bottom band of Figure 7.1.
B	TRUE	A topic is not a storage locker. When a message is published, the broker delivers it to the subscribers of that topic right away. In classic pub/sub the message is broadcast to whoever is currently subscribed and then discarded – there is no waiting around for someone to come and collect it later.
C	FALSE	Direct opposite of D, so exactly one of the pair can be right. Synchronous would mean the publisher blocks until every subscriber has finished processing – which would destroy the whole point of decoupling.
D	TRUE	The publisher fires the message and carries on immediately. It does not wait for subscribers, does not know if they succeeded, and does not care. That is asynchronous.

ANSWER TO QUESTION 8

A, B and D

Matches the options you had selected.

DO B AND D CONTRADICT EACH OTHER? NO – AND THIS IS WORTH UNDERSTANDING

They describe two different sides of the exchange, so both can be true at once:

- **B is about the receiving side:** once published, subscribers are handed the message immediately rather than having to poll for it.
- **D is about the sending side:** the publisher does not block waiting for anyone.

"Immediate delivery" and "the sender does not wait" are perfectly compatible. Think of dropping a letter in the chute of a building: it lands in the letterbox instantly, and you walked away instantly.

7.3 What exactly does a message broker do?

STORY BOX • THE SORTING OFFICE

A broker is a postal sorting office. Its jobs: **read the address and route the letter correctly; try again tomorrow if nobody was home; deliver in the order things were posted; and photocopy a circular for every household on the list** when needed.

The four jobs of a message broker (all four appear in Question 9)

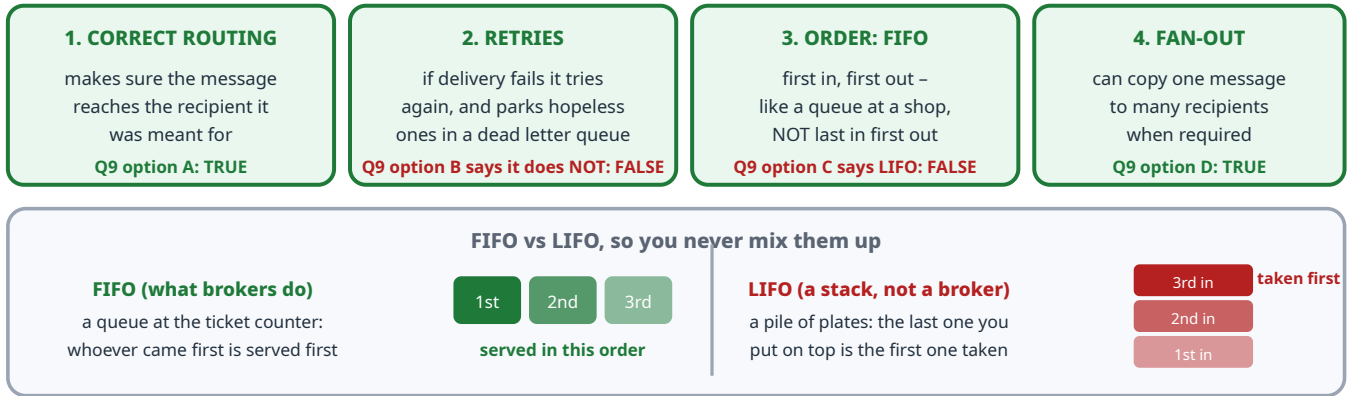


Figure 7.2 – Every option of Question 9 mapped onto a broker's real responsibilities.

Question 9

1 mark • select all that apply

What is the role of the message broker?

- A. It ensures that the message gets delivered to the correct recipient.
- B. If the message is undelivered in the first try, it fails and doesn't retry.
- C. It ensures that the messages are processed in LIFO manner.
- D. It can send messages to multiple recipients, if required.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	Routing to the right recipient is the broker's core job. It is the sorting office reading the address.
B	FALSE	The opposite is true: brokers are built for reliability . They retry failed deliveries, often with increasing gaps between attempts, and move permanently undeliverable messages to a dead letter queue rather than silently losing them. Giving up after one try would make a broker useless.
C	FALSE	Brokers deliver in FIFO order – first in, first out. LIFO is a stack , a completely different data structure. Under LIFO the oldest message could sit unprocessed forever, which is exactly what a queue is designed to prevent.
D	TRUE	"If required" makes it precise and correct. In point-to-point mode one recipient gets the message; in pub/sub mode the broker fans it out to many. Figure 7.1.

ANSWER TO QUESTION 9

A and D

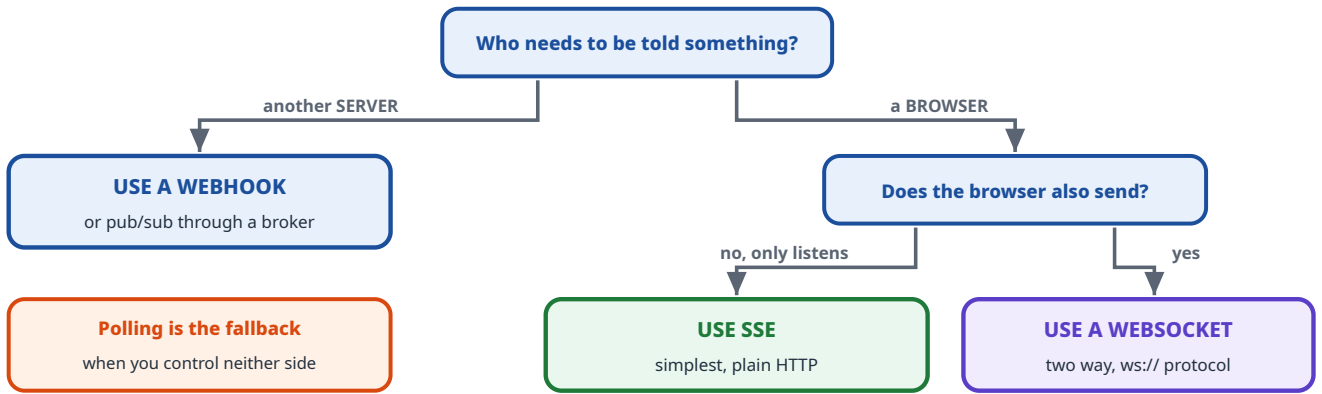
Matches the options you had selected.

All five techniques in one place

If you learn one page from this book, learn this one. Every question in the paper is a row or a column of this table.

	Short polling	Long polling	SSE	WebSocket	Webhook
Who starts	client	client	client opens it	client shakes hands	server (the provider)
Direction	both, one at a time	both, one at a time	one way : server to client	two way (full duplex)	one way : sender to receiver
Connection	opens and closes every time	held open until there is data	stays open for many events	stays open permanently	closes right after the POST
Protocol	HTTP	HTTP	HTTP	<code>ws://</code> after an HTTP upgrade	HTTP
Wasted traffic	very high	low	very low	very low	none
How fresh is the data	up to one interval late	instant	instant	instant	instant
Mainly used between	browser and server	browser and server	browser and server	browser and server	server and server
Sync or async	synchronous request-response	synchronous request, delayed reply	asynchronous stream	asynchronous both ways	asynchronous , event-driven
Good for	simple, rare updates; works anywhere	chat before WebSockets existed	live scores, notifications, progress	chat, games, collaborative editing	payment confirmations, CI triggers
Question in the paper	Q5, Q3	Q6, Q3	Q4	Q2	Q1, Q2, Q7, Q10

A decision tree: which one should I use?



Rule of thumb: use the simplest technique that solves the problem. Do not reach for WebSockets by habit.

Figure 8.1 – *Choosing between the techniques in real projects.*

Answer key, cheat sheet and practice

9.1 Master answer key

Q	Topic	Answer	The one line that decides it
1	Webhooks, general	A, D	Respond immediately; a trivial Flask app is enough. Both wrong options say "cannot".
2	Webhooks vs WebSockets	A, B, C	Webhook = HTTP, server-to-server. WebSocket = 2-way. Only WebSockets stay open.
3	Polling and long poll	A, C	Polling repeats at fixed intervals and needs no persistent connection.
4	SSE (pick the false one)	B	SSE needs an open HTTP stream and a listener – never a task queue.
5	Short polling	A, C	Immediate reply every time, even an empty one, at a fixed interval.
6	Long polling	B	No data means no reply yet; the connection is held open.
7	Webhook vs API	A, D	Webhooks push and are asynchronous ; an API can pull and push. Not C.
8	Pub/Sub	A, B, D	Enables event-driven architecture, delivers to subscribers at once, asynchronous.
9	Message broker	A, D	Correct routing and fan-out. Brokers do retry, and they are FIFO not LIFO.
10	Webhooks, details	A, D	Content in the request body; one-way. Response stays small; no humans involved.

THE SINGLE CHANGE TO MAKE

Question 7: untick option C. Everything else in your attempt already matches this key. If a webhook were synchronous, the receiver would have to be sitting there waiting for it – and the entire reason webhooks exist is so that nobody has to wait.

9.2 The one-page cheat sheet

The core idea

- Plain HTTP is **client-initiated**: the server can never speak first.
- **Pull** = the client asks. **Push** = data arrives unasked.
- All five techniques are workarounds for that one restriction.

Polling

- **Short polling:** fixed repeating interval; server replies **immediately**, even with nothing. Connection closes each time.
- **Long polling:** server **holds the connection open** and stays silent until it has data, then replies.
- Polling does **not** require a persistent connection – each poll is independent.
- Memory hook: *"short reply is instant; long polling makes you wait long."*

Server-Sent Events

- One long-lived HTTP connection; the server streams many events.
- **One way only:** server to client. Mime type `text/event-stream` .
- Client side is the built-in `EventSource` object.
- Needs **no** task queue, **no** broker, **no** new protocol.

WebSockets

- HTTP handshake with `Upgrade: websocket` , then switches to `ws://` .
- **Two way / full duplex**, connection stays open.
- Use when **both** sides need to send: chat, games, collaborative editing.

Webhooks

- A user-defined **HTTP callback**: you register a URL, they POST to it on an event.
- **Push, server-initiated, one way, asynchronous, server-to-server.**
- Content travels in the **request body**; the **response** should be a small, immediate 200.
- Slow response means timeout and **retry**, which can cause duplicate processing.
- Can carry JSON, form-encoded or XML data. Can be secured with HMAC signatures, secret tokens, HTTPS and IP allowlists.
- A receiver can be a trivial Flask route. The connection **closes** immediately.
- Often called a **reverse API**. API = pull (and can push); webhook = push only.

Pub/Sub and brokers

- Publisher sends to a **topic**; every **subscriber** of that topic gets a copy (**fan-out**).
- Publisher does not know who or how many subscribers exist – that is **decoupling**.
- Communication is **asynchronous**; delivery to current subscribers is immediate.
- It is the foundation of **event-driven architecture**.
- Broker jobs: correct **routing**, **retries** plus a dead letter queue, **FIFO** order, and fan-out to **many recipients** when required.
- Queue = one consumer, "who does this job". Pub/Sub = all subscribers, "who needs to know".

9.3 Exam technique: the keyword radar

Seven of the ten questions in this paper are decided by a single word.

Word you spot	What it usually means	Where it happened here
cannot	likely FALSE	Q1 options B and C: "cannot post form encoded data", "cannot be secured".
must	likely FALSE	Q6 option D: "the client <i>must</i> send the next request immediately".
All of the above	check every option first	Q3 and Q5 option D: both die because option B is false.
opposite pairs	exactly one is right	Q5 A vs B, Q8 C vs D, Q3 has the pair split across Q6.
reversed sentences	read the direction twice	Q7 option B swaps push and pull; Q10 option B swaps request and response.
false in the stem	tick the wrong statement	Q4. Underline the word "false" before you start.
if required, in general	likely TRUE	Q9 option D, Q2 option A. Careful hedged wording is usually honest wording.

9.4 Practice questions

Cover the answers. If you can do these eight, you have genuinely learned the topic.

- P1.** A dashboard must show a live share price that changes every second. The browser never sends anything back. Which technique, and why not WebSockets?
- P2.** Your payment provider POSTs to your webhook URL. Your handler takes 45 seconds because it generates an invoice PDF. Name two things that will go wrong, and the fix.
- P3.** A client polls every 10 seconds. An event happens 1 second after a poll. What is the worst-case delay before the client learns about it? What would it be with long polling?
- P4.** Why can a browser tab not be a webhook receiver?
- P5.** Which of these keeps a connection open: short polling, long polling, SSE, WebSocket, webhook?
- P6.** One message is published to a topic with 4 subscribers. How many times is it processed? And if it were sent to a point-to-point queue with 4 consumers?
- P7.** Your webhook URL is public. Anyone can POST fake payment events to it. How do you stop them?
- P8.** Give one reason to choose SSE over WebSockets, and one reason to choose WebSockets over SSE.

ANSWERS TO THE PRACTICE QUESTIONS

- P1. SSE.** The data flows one way only, so a two-way channel would be wasted complexity: WebSockets need protocol support on the server, extra infrastructure configuration, and they lose the free automatic reconnection that `EventSource` gives you. Use the simplest tool that fits.
- P2.** (1) The provider **times out** and marks the delivery as failed. (2) It then **retries**, so you may process the same payment two or three times and send duplicate invoices. The fix: reply **200 OK immediately**, push the PDF job onto a background queue, and make your handler **idempotent** so a repeated event is harmless.
- P3.** Polling: worst case about **9 seconds** (it must wait for the next scheduled poll). Long polling: **essentially zero** – the connection is already open and silent, so the server responds the instant the event happens.
- P4.** A webhook is an **incoming** HTTP POST, so the receiver needs a **publicly reachable URL and a server listening on it**. A browser tab has no public address and cannot accept incoming connections. That is why webhooks are server-to-server.
- P5.** **Long polling** (holds one request open), **SSE** and **WebSocket** (both truly persistent). **Short polling** and **webhooks** close the connection immediately.
- P6.** Topic with 4 subscribers: **4 times**, once per subscriber (fan-out). Point-to-point queue with 4 consumers: **once** – the first consumer to take it removes it, and the other three never see it.
- P7.** Verify an **HMAC signature** computed from the body with a shared secret, so a forged POST fails the check. Add HTTPS, a secret token, a timestamp check to block replays, and optionally an **IP allowlist**. Reject anything that fails.
- P8.** **SSE over WebSocket:** it is plain HTTP, so it works through ordinary proxies and firewalls with no special server support, and it reconnects automatically. **WebSocket over SSE:** the client can send messages too, which SSE simply cannot do.

9.5 Glossary

Term	Meaning in one sentence
Client-initiated	Nothing happens until the client sends a request; the normal HTTP rule.
Pull	The receiver asks for data when it wants it.
Push	Data arrives without the receiver having just asked.
Short polling	Repeating requests at a fixed interval, each answered immediately.
Long polling	A request the server holds open, unanswered, until it has data.
Persistent connection	One connection deliberately kept alive across many messages.
SSE	Server-Sent Events: a one-way HTTP stream of events from server to client.
text/event-stream	The content type that marks an HTTP response as an SSE stream.

EventSource	The browser object that opens and listens to an SSE stream.
WebSocket	A persistent two-way connection using the <code>ws://</code> protocol.
Full duplex	Both sides can send at the same time, independently.
Handshake	The opening exchange where client and server agree to upgrade the connection.
Webhook	A user-defined HTTP callback: they POST to your URL when an event happens.
Reverse API	Nickname for a webhook, because they call you instead of you calling them.
Request body	The data carried inside an HTTP request; where webhook content lives.
HMAC signature	A hash of the body made with a shared secret, proving the sender is genuine.
Idempotent	Safe to run more than once with the same result; vital for webhook retries.
Event-driven	Work is triggered by events happening, not by anyone asking.
Synchronous	Send and wait for the reply before continuing.
Asynchronous	Send and carry on immediately; the reaction happens later.
Publisher	The sender in pub/sub, who writes to a topic.
Subscriber	A receiver that has registered interest in a topic.
Topic	A named channel; messages sent to it go to all its subscribers.
Fan-out	Copying one message to many recipients.
Message broker	The middleman service that stores, routes and retries messages.
Point-to-point	One message consumed by exactly one consumer.
FIFO	First in, first out: a queue, like a line at a counter.
LIFO	Last in, first out: a stack, like a pile of plates. Not how brokers work.
Dead letter queue	Where a broker parks messages it could never deliver.
Decoupling	Arranging things so two parts need not know about each other.

9.6 Final word

IF YOU REMEMBER ONLY SIX SENTENCES FROM THIS WHOLE BOOK

1. HTTP makes the **client speak first**; every technique here works around that.
2. **Short** polling answers instantly even with nothing; **long** polling holds the connection open and stays silent.
3. **SSE is one way** (a loudspeaker); a **WebSocket is two way** (a phone call).
4. A **webhook** is push, server-initiated, one way, **asynchronous**, and server-to-server; it closes immediately and needs a fast 200.
5. An **API can pull and push**; a **webhook only pushes**.
6. **Pub/Sub** fans one message out to every subscriber of a topic, and a **broker** routes, retries and works **FIFO**.

Prepared as a step-by-step teaching solution.

Work through the figures with a pencil in hand; that is where the learning actually happens.

NOTES ON SOURCES

Technical facts were checked against the official documentation for each technology: [MDN on Server-Sent Events and EventSource](#), [MDN on the WebSockets API](#), and [the Flask documentation](#). The characterisation of webhooks as **push-based and event-driven**, in contrast to the request-response pull model of an API, was cross-checked across multiple independent technical references, which describe the distinction consistently. The pub/sub delivery semantics (immediate fan-out to current subscribers, versus a queue that stores a message for a single consumer) were checked against broker documentation and messaging-pattern references. All explanations, analogies and diagrams here were written and drawn for this document. Content was rephrased for compliance with licensing restrictions.